Question 3

Two main measures for the efficiency of an algorithm are:

- ○ Processor and memory
- ○ Complexity and capacity
- ◉ Time and space
- ○ Data and space

Question 8

The asymptotic relationship between $f(n)=\log n^{\log 17}$ and $g(n)= \log 17^{\log n}$ is:

- ○ $f(n) = O(g(n))$
- ○ $f(n) = \Omega(g(n))$
- ○ $f(n) = \Theta(g(n))$
- ◉ All the above are correct

Question 4

For any two algorithms f(n) and g(n), either f(n) = O(n) or g(n) = O(f(n)).

- ○ True
- ◉ False

Question 7

Consider the recurrence: $T(n) = 2T(n/4) + \sqrt{n}$, using the **Master method**, choose the correct answer.

- ○ Apply case 1 and $T(n) = \Theta(\sqrt{n})$
- ◉ Apply case 2 and $T(n) = \Theta(\sqrt{n}.\log n)$
- ○ Apply case 3 and $T(n) = \Theta(\sqrt{n})$
- ○ Master method cannot be used.

Question 9

If $f(n) * g(n) = O(n^3)$, then:

- ○ $f(n) = g(n) = n^3$
- ○ $f(n) \neq n^3$ and $g(n) \neq n^3$
- ◉ $f(n) = n$ and $g(n) = n^2$
- ○ $f(n) = n^3$ and $g(n) = n$

Question 11

Using the **recursion tree** method to solve **T(n) = T(n/10) + c** and **T(1) = 1**, the total cost of all subproblems in the last level is:

- (●) 1
- ( ) n . c
- ( ) c
- ( ) n

Question 6

Using the **recursion tree** method to solve **T(n)=T(n/10)+c** and **T(1)= 1**, then the total cost of all levels is:

- $O(c.\log_{10}n + 1)$
- $O(\log_{10}n)$
- $O(n.\log_{10}n)$
- (selected) Both **a** and **b** are correct

Question 10

The substitution method comprises two steps, guessing the form of the solution and using induction to show that the guess is valid.

- (●) True
- ( ) False

Question 12

Using the **substitution method** to solve **T(n) = T(n-2) + 2** and **T(1) = 1**, the solution guess is:

- ○ T(n) = log n
- ◉ T(n) = n
- ○ T(n) = log n +1
- ○ T(n) = n.logn

Question 14

For all positive f(n), g(n), and h(n), if f(n) = O(g(n)) and f(n) = Ω(h(n)), then g(n) = Ω (h(n)).

- (●) True
- ( ) False

Question 16

Based on the time complexity, which of the following algorithms you will choose?

- ○ **Algorithm A** solves a problem of size n by recursively solving 2 subproblems of size n/2 and then combining their solutions in time O(n).
- ◉ **Algorithm B** solves a problem of size n by recursively solving 2 subproblems of size n/2 and then combining their solutions in time O(1).
- ○ **Algorithm C** solves a problem of size n by recursively solving 2 subproblems of size 2n and then combining their solutions in time O(n).
- ○ Cannot be determined.

Question 15

To solve **T(n)= T(n-2) + 2 and T(1) =1** using the **substitution method**, we need to assume that the guess is true for:

- ○ k/2
- ○ k-1
- ◉ k-2
- ○ k

Question 17

Consider the recurrence: **T(n)= T(n) + 1**, using the **Master method**, choose the correct answer.

- ○ Apply case 1 and $T(n) = \Theta(1)$
- ○ Apply case 2 and $T(n) = \Theta(\log n)$
- ○ Apply case 3 and $T(n) = \Theta(1)$
- ◉ Master method cannot be used.

Question 18

Could the **substitution method** be used to prove that $T(n) = 2T(n/2) + n$ and $T(1) = 1$ is $O(n \log(n) + n)$?

- ○ True
- ◉ False

Question 13

Using the **recursion tree** method to solve **T(n) = T(n/10) + c** and **T(1) = 1**, the cost of each subproblem in the last level is:

- (●) 1
- ( ) n . c
- ( ) c
- ( ) n

Question 5

Consider **missingNumber** algorithm that returns the missing number in an array **A** of n sorted integers in the range of 1 to n+1.

```
Algorithm missingNumber(A[], i, j)
if (i >= j)
        if (i == A[i])
            return i+1
        else
            return i
m = (i+j)/2
if (A[m] > m)
        return missingNumber(A, i, m)
else
        return missingNumber(A, m+1,j)
```

Choose the function that represents the running time of **missingNumber** algorithm.

- ○ T(n) =2T(n/2) + 1 , T(1) =1
- ◉ T(n) = T(n/2) + 1, T(1) = 1
- ○ T(n) = T(n/2) + n, T(1) = 1
- ○ T(n) = T(n/2) + 1, T(1) = n

Question 20

Using the **recursion tree** method to solve **T(n) = T(n/10) + c** and **T(1) = 1**, the exact total cost of subproblems in each level except the last one is:

- 1
- c
- $c/10^1$
- 10c

Remaining Time: 43 minutes, 38 seconds.

Question Completion Status:

⚠ Moving to another question will save this response.

Question 9

For any two algorithms f(n) and g(n), either $f(n) = O(n)$ or $g(n) = O(f(n))$.

- ○ True
- ◉ False

⚠ Moving to another question will save this response.

o search

DELL

الرجاء
التاكد من
الاجوبة